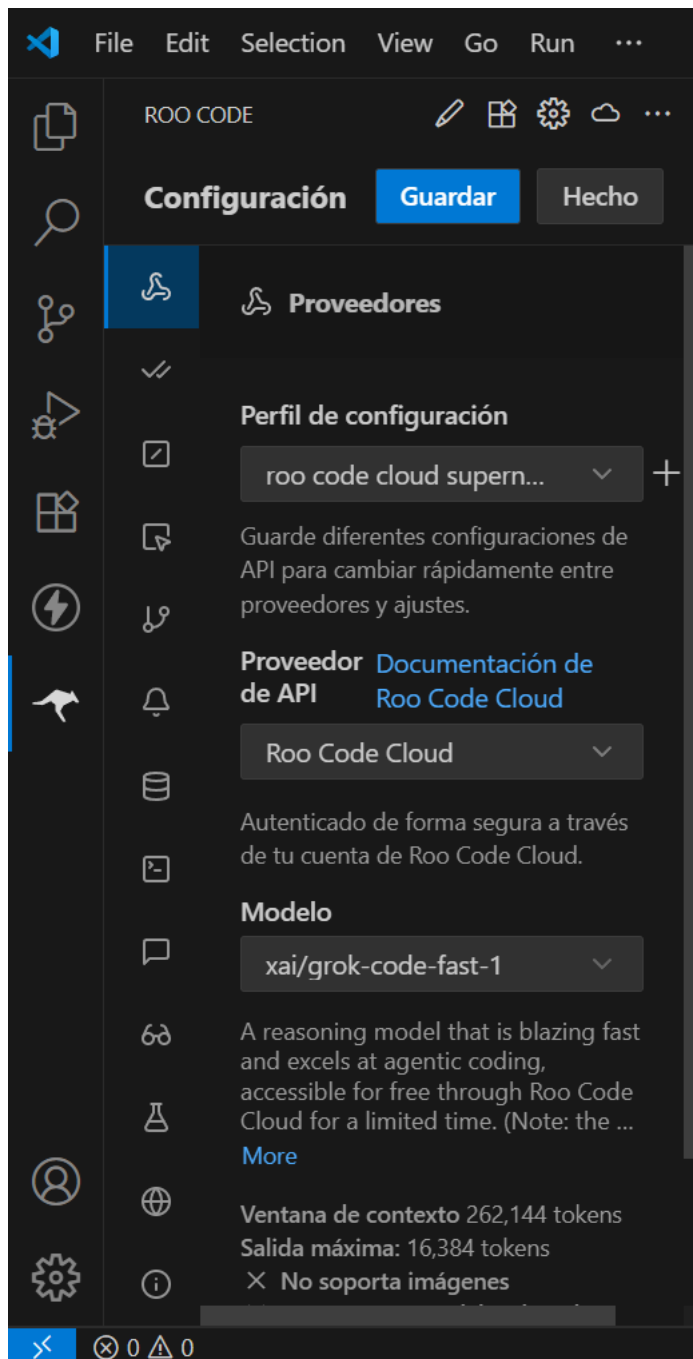


1.



2. Registro de Prompts (40%)

Paso	Prompt usado	Resultado clave obtenido	Justificación
1.1 (Architect)	“Diseña un modelo de datos para un sistema de gestión de citas simples que evite duplicados de fecha y hora.”	Roo Code generó un esquema con la tabla Appointment(id, client_name, date, time, service_type) e incluyó la validación de duplicados como requerimiento.	El prompt fue efectivo porque dio un modelo exacto y sencillo ajustado a los requisitos del reto.
1.2 (Architect)	“Genera un diagrama de alto nivel del sistema Appointment Booker indicando frontend, backend y base de datos.”	Roo Code produjo un diagrama lógico con tres capas: Frontend (HTML/JS) → Backend (Node.js/Express) → DB (Supabase/Postgres) .	Ayudó a visualizar claramente la arquitectura antes de pasar a la implementación.
1.3 (Architect)	“Dame un plan paso a paso para implementar este sistema en Node.js con Express y Supabase.”	Obtuve un roadmap de 5 pasos: configurar Express, conectar DB, crear endpoints, validar duplicados, probar en Postman y luego crear el frontend básico.	Fue útil porque proporcionó una guía secuencial clara para implementar sin saltos.
2.1 (Code/Developer)	“Crea el código backend en Node.js con Express que maneje citas con validación	Roo Code devolvió un archivo server.js con conexión a Postgres, endpoint /appointments y validación de duplicados.	Fue el más efectivo porque entregó el esqueleto funcional listo para pruebas.

Paso	Prompt usado	Resultado clave obtenido	Justificación
	de duplicados en Supabase.”		
2.2 (Code/Developer)	“Optimiza el código para manejar errores y devolver mensajes claros al cliente.”	Roo Code mejoró el manejo de errores (status 400 y 500 con mensajes JSON claros).	Este ajuste fue crucial para robustecer el backend y hacerlo más profesional.
2.3 (Code/Developer)	“Genera un formulario HTML simple que permita crear citas y enviarlas al backend.”	Roo Code generó un archivo index.html con campos (nombre, fecha, hora, servicio) y un fetch() al endpoint.	Fue efectivo porque dio la interfaz mínima requerida por el reto.

3. Análisis de Uso (20%)

El Modo Architect permitió obtener una visión general y estructurada del sistema, ideal para planificar el diseño antes de escribir código. En cambio, el Modo Code/Developer fue mucho más práctico para generar directamente el backend y frontend, resolviendo problemas específicos de sintaxis y validaciones. Ambos se complementaron muy bien.

4. Código Final (40%)

```
const express = require('express');

const { createClient } = require('@supabase/supabase-js');

const cors = require('cors');

require('dotenv').config();

const app = express();

const port = 3000;
```

```
// Inicializar cliente de Supabase

const supabaseUrl = process.env.SUPABASE_URL;
const supabaseKey = process.env.SUPABASE_ANON_KEY;
const supabase = createClient(supabaseUrl, supabaseKey);
app.use(express.json());
app.use(cors());
```

```
// Ruta de prueba

app.get('/', (req, res) => {
  res.send('API Appointment Booker funcionando');
});
```

```
// Middleware para manejo de errores general

app.use((err, req, res, next) => {
  console.error('Error no manejado:', err);
  res.status(500).json({
    error: 'Error interno del servidor',
    message: 'Ocurrió un error inesperado en el servidor'
  });
});
```

```
// Ruta POST para crear una nueva cita

app.post('/appointments', async (req, res) => {
  const { client_name, date, time, service_type } = req.body;

  // Validación de campos requeridos
```

```
if (!client_name || !date || !time || !service_type) {  
  return res.status(400).json({  
    error: 'Datos incompletos',  
    message: 'Todos los campos son requeridos: client_name, date, time,  
service_type',  
    required_fields: ['client_name', 'date', 'time', 'service_type']  
  });  
}
```

```
try {  
  // Verificar si ya existe una cita en la misma fecha y hora  
  const { data: existingAppointment, error: fetchError } = await supabase  
    .from('appointments')  
    .select('id')  
    .eq('date', date)  
    .eq('time', time)  
    .single();  
  
  if (existingAppointment) {  
    return res.status(409).json({  
      error: 'Cita duplicada',  
      message: 'Ya existe una cita para esta fecha y hora',  
      conflict_date: date,  
      conflict_time: time  
    });  
  }  
}
```

```

// Insertar la nueva cita

const { data, error, status } = await supabase
  .from('appointments')
  .insert([
    { client_name, date, time, service_type }
  ])
  .select();

if (error) {
  console.error('Error al crear cita:', error);

  // Verificar si es un error de restricción UNIQUE
  if (error.code === '23505') {
    return res.status(409).json({
      error: 'Cita duplicada',
      message: 'Ya existe una cita para esta fecha y hora',
      conflict_date: date,
      conflict_time: time
    });
  }

  return res.status(status || 500).json({
    error: 'Error al crear cita',
    details: error.message
  });
}

res.status(201).json({
  message: 'Cita creada exitosamente',
  appointment: data[0]
});

```

```

    } catch (err) {
      if (err.message.includes('already exists')) {
        // Capturar el error de duplicado de UNIQUE constraint
        return res.status(409).json({
          error: 'Cita duplicada',
          message: 'Ya existe una cita para esta fecha y hora',
          conflict_date: date,
          conflict_time: time
        });
      }
    }

    console.error('Error en el servidor:', err);
    res.status(500).json({
      error: 'Error interno del servidor',
      message: 'Ocurrió un error al procesar la solicitud'
    });
  }
});

// Ruta GET para listar citas
app.get('/appointments', async (req, res) => {
  try {
    const { data, error, status } = await supabase
      .from('appointments')
      .select('*');

    if (error) {

```

```

    console.error('Error al obtener citas:', error);

    return res.status(status || 500).json({
      error: 'Error al obtener citas',
      details: error.message
    });
  }

  res.json(data);
} catch (err) {
  console.error('Error en el servidor:', err);
  res.status(500).json({
    error: 'Error interno del servidor',
    message: 'Ocurrió un error al procesar la solicitud'
  });
}
});

app.listen(port, () => {
  console.log(`Servidor escuchando en http://localhost:${port}`);
});

```

